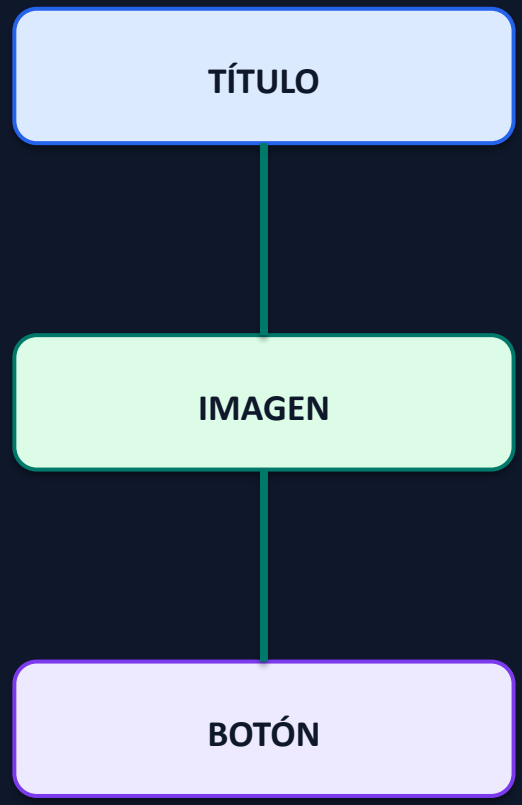
ConstraintLayout en Android Studio

De colocar elementos en orden a definir relaciones entre Views

Docente: José Manuel Villar Ferradal



Objetivos de aprendizaje

Primera práctica completa con ConstraintLayout.

Comprender

Qué es una constraint y cómo cambia el modelo mental.

Relacionar

TOP, BOTTOM, START y END con parent u otra View.

Dimensionar

wrap_content, tamaños fijos y 0dp / match constraints.

Construir

Una ficha de producto completa sin anidar layouts.

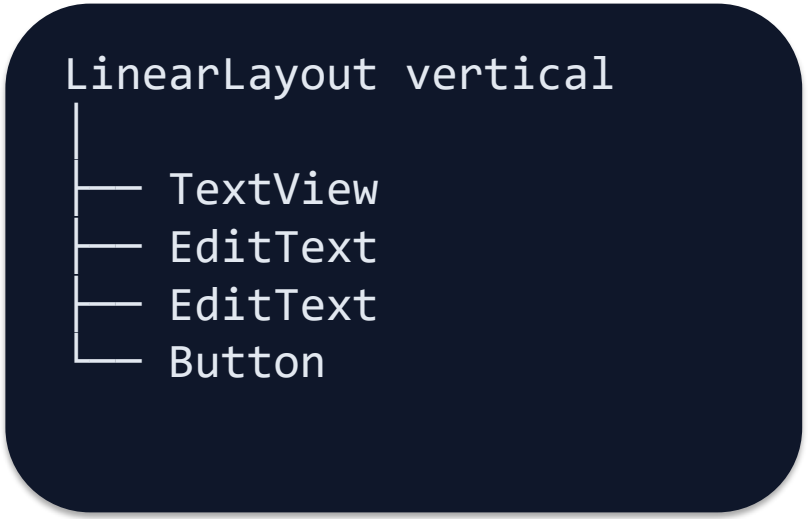
Comparar

Cuándo usar LinearLayout y cuándo ConstraintLayout.

Antes: ¿cómo pensábamos con LinearLayout?

LinearLayout

El orden de los hijos y la orientación determinan gran parte de la colocación.



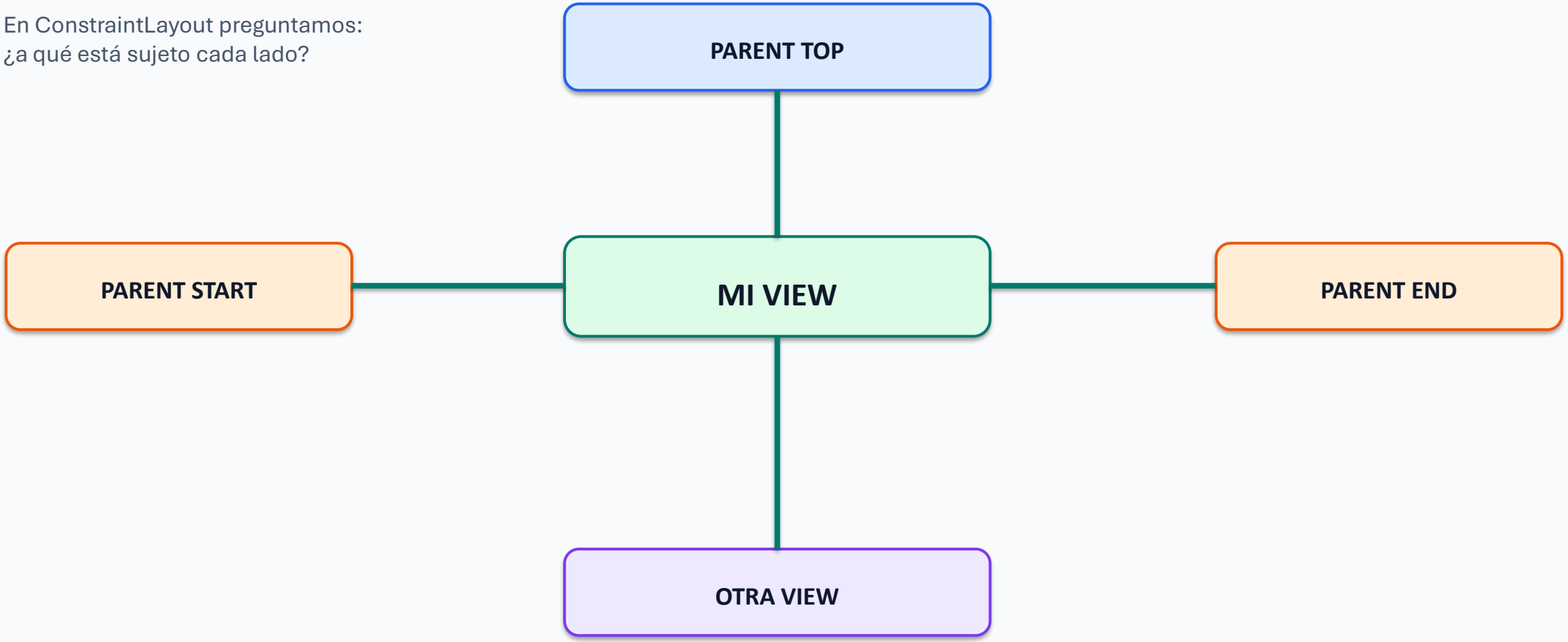
Pregunta

¿Qué ocurre cuando queremos relaciones más específicas entre elementos?

- Botón pegado abajo.
- Imagen centrada entre dos límites.
- Texto alineado con otra View.
- Evitar varios layouts anidados.

El nuevo modelo mental

En ConstraintLayout preguntamos:
¿a qué está sujeto cada lado?



“¿Qué lado mío conecto con qué lado de qué elemento?”

Estructura XML básica

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- Views -->

</androidx.constraintlayout.widget.ConstraintLayout>
```

android:

Atributos estándar de Android.

app:

Aquí veremos los atributos de constraints.

Los cuatro lados que usaremos

TOP

parte superior

BOTTOM

parte inferior

START

inicio horizontal

END

final horizontal

Regla práctica

En español y otros idiomas LTR:
START ≈ izquierda
END ≈ derecha

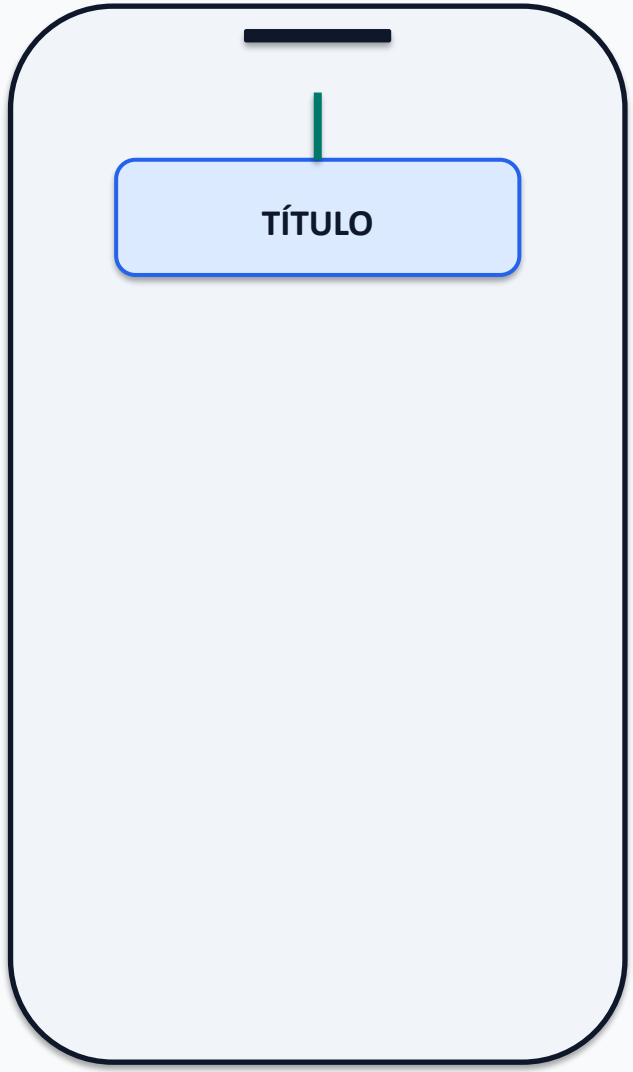
Pero aprenderemos START/END porque se adaptan a la dirección del idioma.

Primera constraint: TOP con TOP del parent

```
<TextView
    android:id="@+id/txtTitulo"
    ...
    app:layout_constraintTop_toTopOf="parent" />
```

Leer el atributo

Mi TOP → TOP del parent

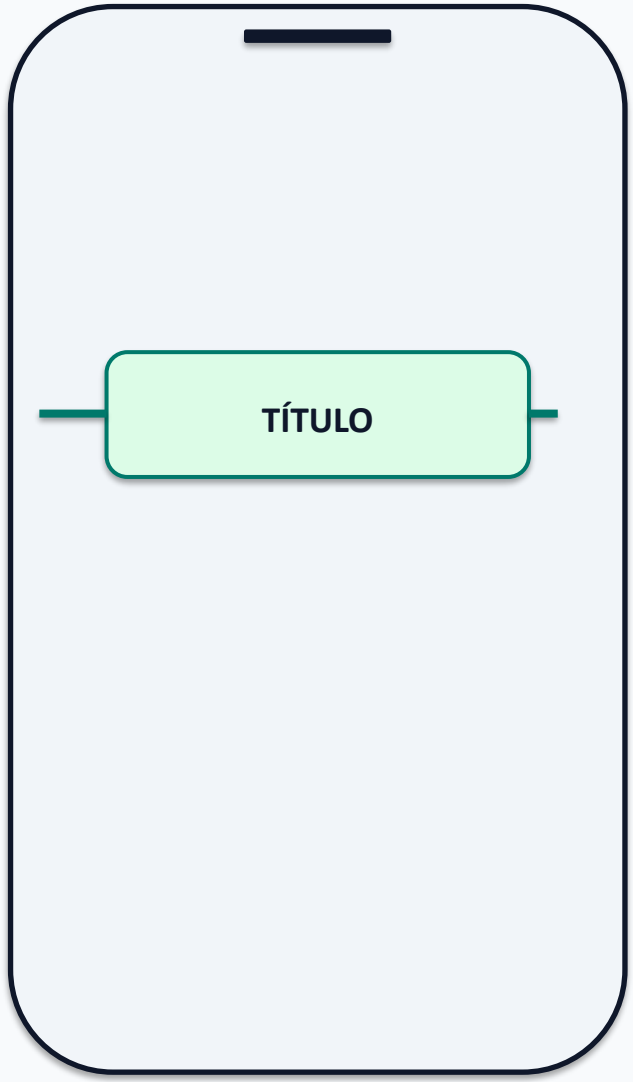


Centrado horizontal con START + END

```
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"
```

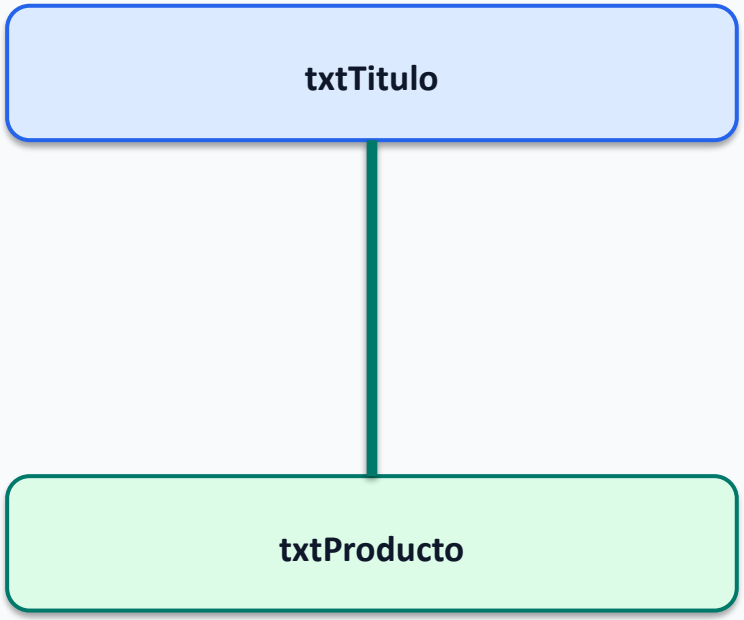
¿Por qué se centra?

La View queda restringida por ambos lados. Con el bias horizontal por defecto (50 %), queda centrada.



Restricciones entre Views

Aquí está la potencia principal de ConstraintLayout.




```
app:layout_constraintTop_toBottomOf="@id/txtTitulo"
```

Mi TOP se conecta con el BOTTOM de txtTitulo

Combinaciones frecuentes

No hace falta memorizarlas: hay que aprender a leerlas.

Top → Top

app:layout_constraintTop_toTopOf="parent"

Top → Bottom

app:layout_constraintTop_toBottomOf="@id/otraView"

Start → Start

app:layout_constraintStart_toStartOf="parent"

End → End

app:layout_constraintEnd_toEndOf="parent"

Bottom → Bottom

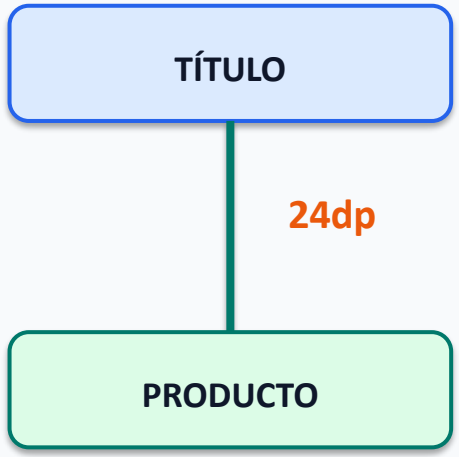
app:layout_constraintBottom_toBottomOf="parent"

Start → End

app:layout_constraintStart_toEndOf="@id/otraView"

Constraints posicionan; margin separa

```
android:layout_marginTop="24dp"
app:layout_constraintTop_toBottomOf="@id/txtTitulo"
```



Importante

El margin no sustituye a una constraint. Primero definimos la relación; después añadimos separación.

Anchura y altura en ConstraintLayout

wrap_content

La View mide lo que necesita su contenido.

Tamaño fijo

Ej.: 160dp. Útil cuando el diseño requiere una medida concreta.

0dp

Match constraints: usa el espacio delimitado por las constraints.

```
android:layout_width="0dp"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"
```

0dp NO significa ancho cero

En ConstraintLayout significa “ajústame al espacio disponible entre constraints”.

Visualizando 0dp / match constraints

Con 0dp

El ancho lo determinan las restricciones laterales.



Con wrap_content

El ancho lo determina el contenido.

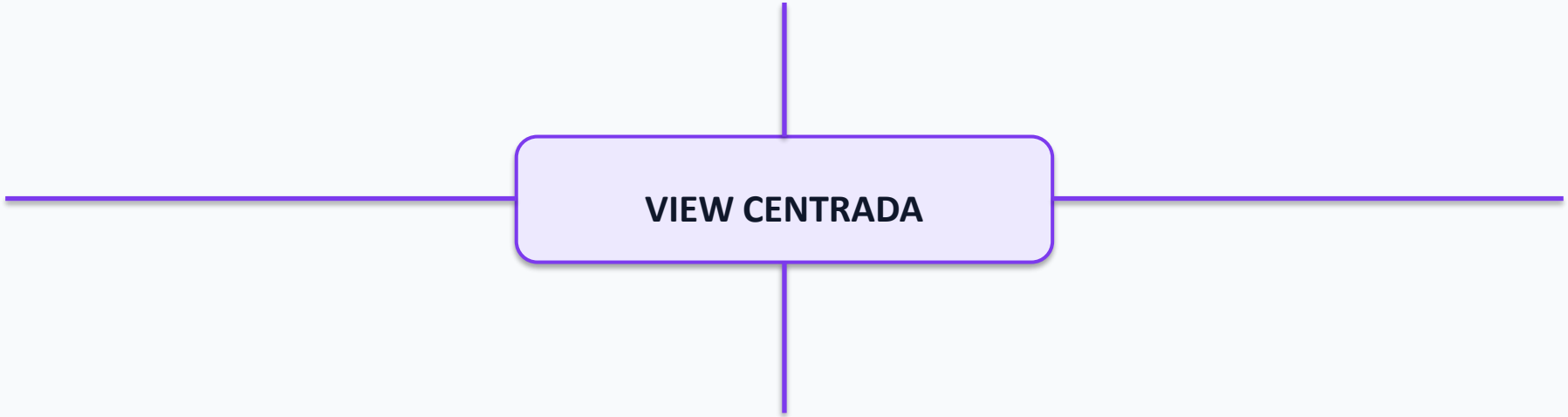
Centrado mediante constraints

Centrado horizontal

START → START del parent
END → END del parent

Centrado vertical

TOP → TOP del parent
BOTTOM → BOTTOM del parent



Bias: cuando está sujeto a ambos lados

Lo mencionamos ahora; lo profundizaremos en una práctica posterior.

```
app:layout_constraintHorizontal_bias="0.5"
```

0.5

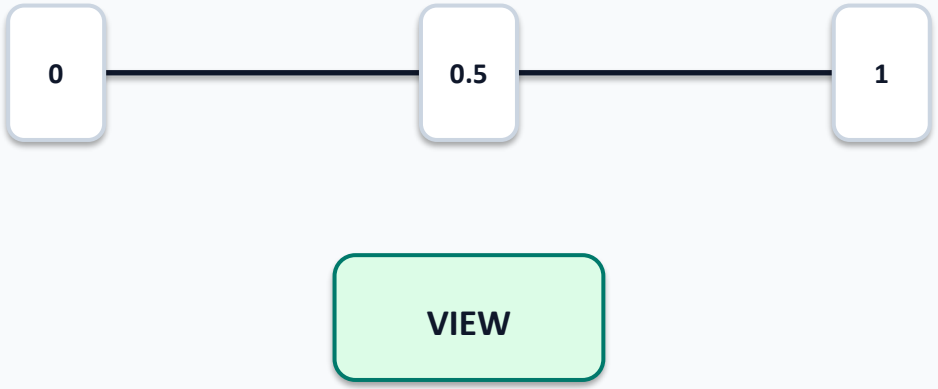
Centro (valor por defecto).

0.0

Hacia START.

1.0

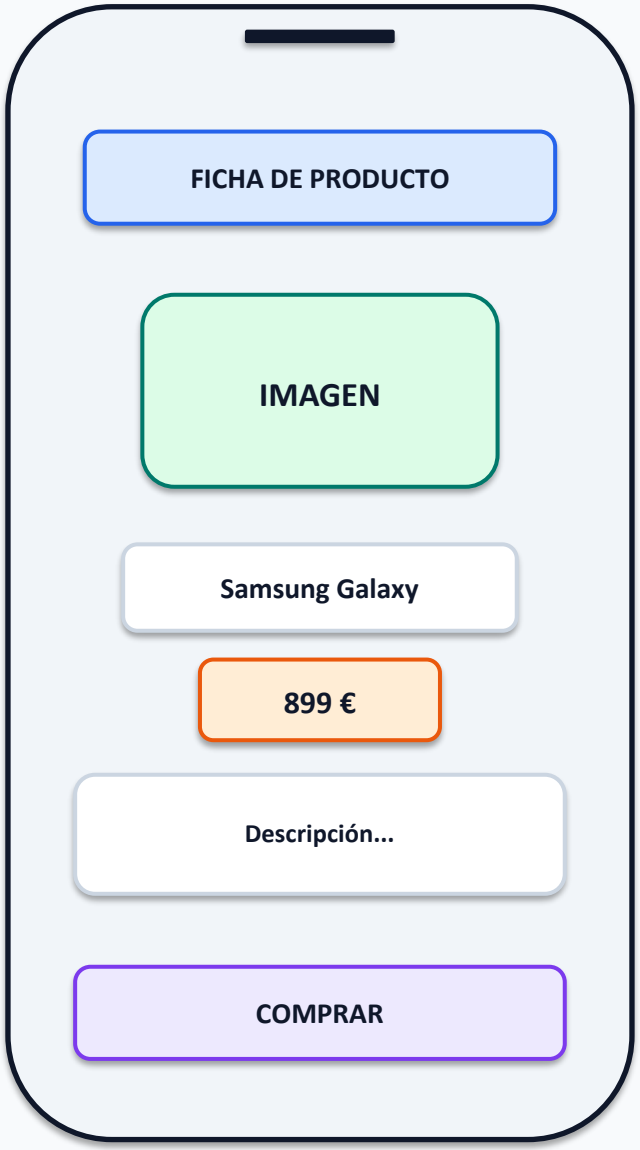
Hacia END.



Práctica: ficha de producto

Construiremos esta pantalla con un único ConstraintLayout.

- Título centrado arriba.
- Imagen debajo del título.
- Nombre y precio relacionados con la imagen.
- Descripción a ancho disponible.
- Botón anclado a la parte inferior.



Paso 1 · Título

```
<TextView
    android:id="@+id/txtTitulo"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="32dp"
    android:text="Ficha de producto"
    android:textSize="26sp"

    app:layout_constraintTop_toTopOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Tres constraints

TOP + START + END

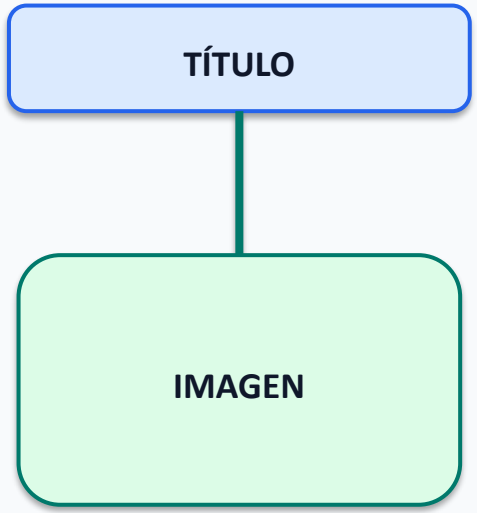
Resultado

Arriba y centrado horizontalmente.

Paso 2 · Imagen debajo del título

```
<ImageView
    android:id="@+id/imgProducto"
    android:layout_width="160dp"
    android:layout_height="160dp"
    android:layout_marginTop="24dp"
    android:contentDescription="Imagen del producto"

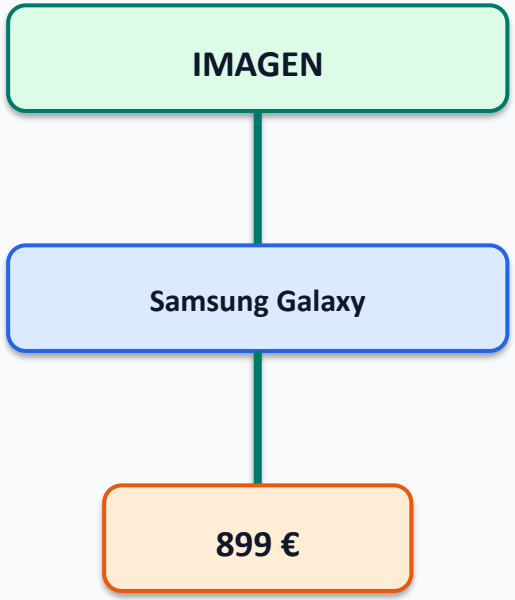
    app:layout_constraintTop_toBottomOf="@id/txtTitulo"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```



Paso 3 · Nombre y precio

```
<!-- Nombre -->
app:layout_constraintTop_toBottomOf="@id/imgProducto"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"

<!-- Precio -->
app:layout_constraintTop_toBottomOf="@id/txtProducto"
app:layout_constraintStart_toStartOf="parent"
app:layout_constraintEnd_toEndOf="parent"
```



Paso 4 · Descripción a ancho disponible

```
<TextView
    android:id="@+id/txtDescripcion"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="24dp"
    android:layout_marginTop="24dp"
    android:layout_marginEnd="24dp"
    android:text="Smartphone Android con..."

    app:layout_constraintTop_toBottomOf="@id/txtPrecio"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent" />
```

Concepto nuevo

width = 0dp

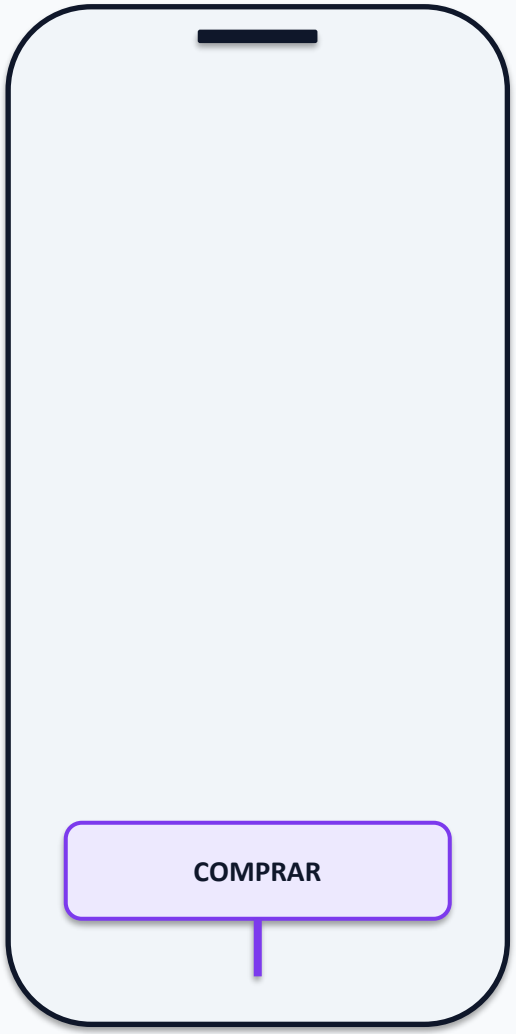
¿Qué consigue?

Ocupa el ancho disponible entre START y END respetando los márgenes.

Paso 5 · Botón anclado abajo

```
<Button
    android:id="@+id/btnComprar"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="24dp"
    android:layout_marginEnd="24dp"
    android:layout_marginBottom="24dp"
    android:text="Comprar"

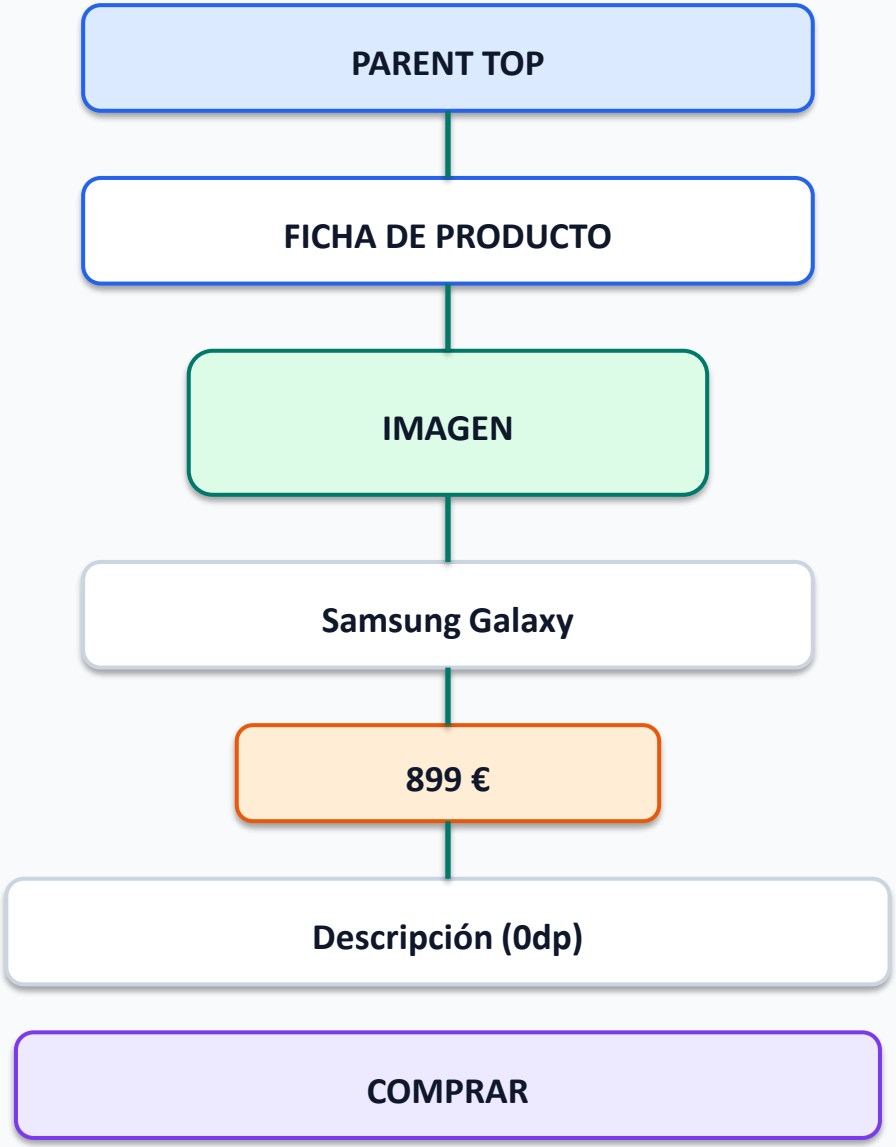
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintBottom_toBottomOf="parent" />
```



La pantalla es una red de relaciones

Lectura vertical

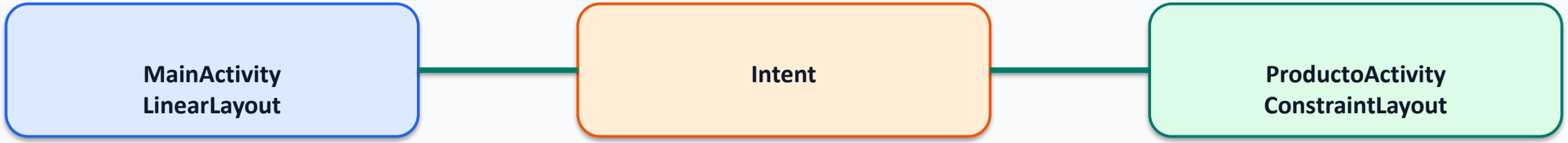
Cada elemento conoce su relación con el anterior o con el parent.



Lectura horizontal

START y END controlan alineación, centrado y ancho disponible.

Conectamos esta Activity con el proyecto anterior

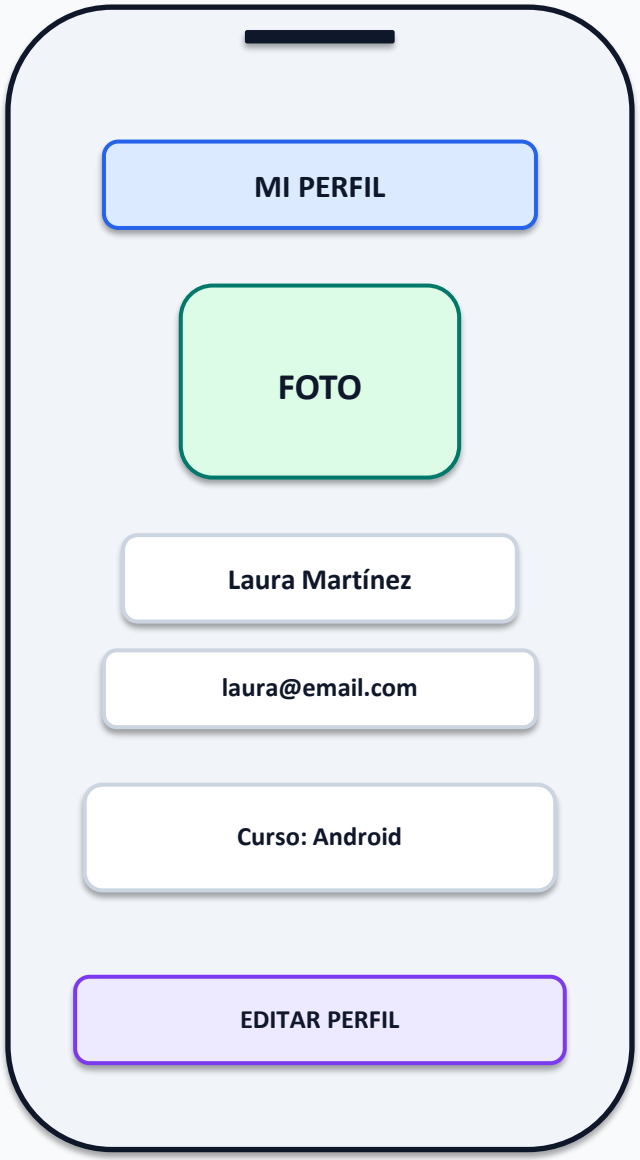



```
val boton = findViewById<Button>(R.id.btnVerProducto)

boton.setOnClickListener {
    val intent = Intent(this, ProductoActivity::class.java)
    startActivity(intent)
}
```

Ejercicio individual · Mi perfil

- Un único ConstraintLayout.
- Sin LinearLayout anidados.
- Usar parent y @id/otraView.
- Aplicar TOP, BOTTOM, START y END.
- Usar al menos una View con width=0dp.
- Antes de programar: dibujar las constraints.

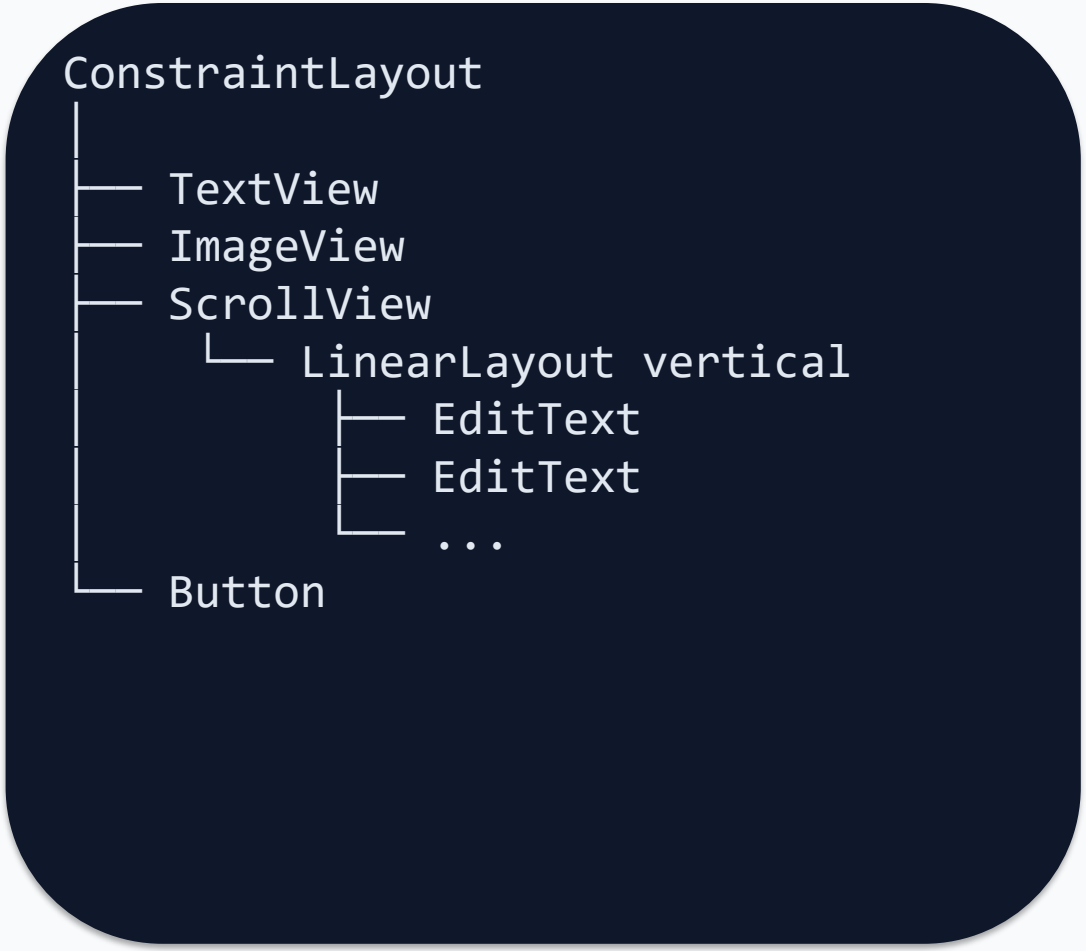


LinearLayout vs ConstraintLayout

	LinearLayout	ConstraintLayout
Idea principal	Secuencia	Relaciones
Dirección	Vertical / horizontal	Constraints
Grupos simples	Muy cómodo	También posible
Pantallas complejas	Puede requerir anidamientos	Suele reducir anidamientos
Aprendizaje	Más inmediato	Más flexible, requiere razonamiento

¿Se pueden combinar layouts? Sí

Cada contenedor puede resolver una parte distinta de la interfaz.



Buena combinación

ConstraintLayout para la estructura general.

LinearLayout

Para grupos sencillos en fila o columna.

ScrollView

Cuando el contenido supera la altura disponible.